

Can a small model solve harder problems by dividing the work?

Preliminary research and questions for discussion

We can teach a useful routine.

Training improved how the model gathers information and calculates an answer.

The connections also matter.

A good answer to a small question can be lost or misused when answers are combined.

Our next goal is to turn these observations into a narrow, reproducible research contribution.

Alex Towell | Advisor discussion | These are preliminary findings, not a general solution.

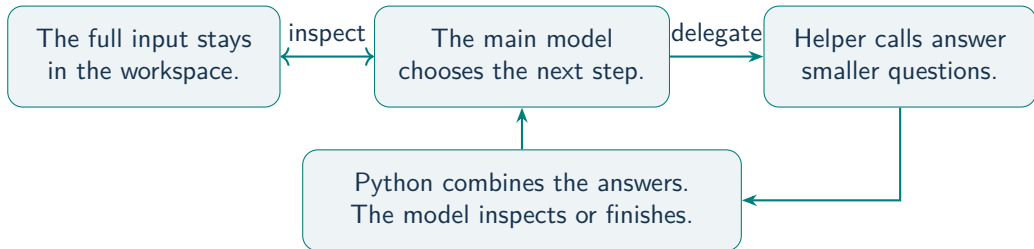
Exploratory research

11 September 2026

1/14

An RLM lets a language model use Python and ask smaller questions.

A recursive language model (RLM) writes code to inspect its input, ask helpers, and combine their answers.



The full input can be too large for one model call. A helper can divide its own task again.

Python runs the model's code. Our tests mostly use one layer of helper calls.

Concept: [Alex Zhang's blog](#) and [Recursive Language Models](#).

The helper reads the text; Python does the counting.

For each user, add the weights of their place questions. How many users have a total above 5?

User	Question	Weight	Place question?
Ada	Where is Oslo?	4	Yes
Ada	What is the capital of Peru?	3	Yes
Ben	Who wrote Hamlet?	6	No
Ben	Where is Kyoto?	2	Yes

Python computes Ada: $4 + 3 = 7$ and Ben: 2.

Only Ada exceeds 5, so the answer is **1 user**.

Weights are invented test values, not confidence scores. This is a simplified illustration, not a quoted training example.

We taught the main model what to do next, not just what answer to give.

In supervised fine-tuning (SFT), demonstrations show what the model sees and what Python code it should write next.

An example observation

“Count users whose place-question weights total more than 5.”

The records are stored in a file the model can read.

What it learns to do

Ask the helper which questions concern places.

Add their weights for each user.

Count totals above 5 and return the answer.

Simplified example. Training used 72 demonstrations, repeated six times, including real helper replies and Python results. This teaches a routine, not open-ended planning.

Training improved performance on newly selected inputs.

We trained two copies of the same model on different demonstrations.



Both improved on the same 72 questions built from eight new sets of records.

We checked the calculation as well as the answer. The helper stayed fixed; the question types were familiar. Missing results allow totals of 12–19, 55, and 53–55 successes. [S3]

An answer can be right for the wrong record.

We next tested the helper alone: does a passage support, contradict, or leave a statement unclear?

A simple reading task

Passage: “Maya bought a red bike.”

Row 7: “Maya bought a vehicle.”

The passage supports this statement.

Row 8: “The bike is blue.”

The passage contradicts this statement.

Suppose row 7’s answer is stamped “row 8.”

The model may answer for row 8 instead.

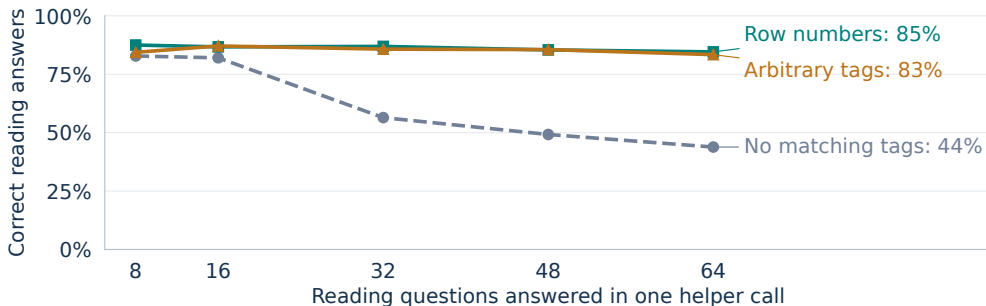
“Contradicted” is then a sensible label—attached to the wrong statement.

Accuracy was 39% with wrong-row names and 80% with names pointing to no row.

Simplified example of deliberately misleading names. Software fixed each answer’s name; the model chose the label. The comparison used 16 batches of 48 questions. [H1]

Matching tags kept answers accurate as the helper handled more records at once.

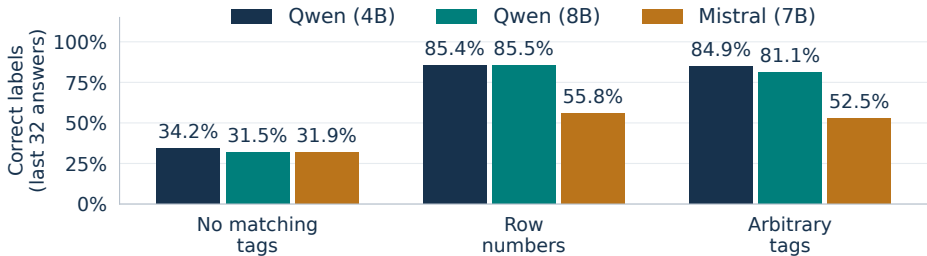
A tag is a name repeated beside a record and its answer, such as “row 7.”



One model, 16 new sets of records, no retraining. Each point includes every answer in its batch. Software supplied tags and order, not reading labels. All 240 outputs followed the format. [H8]

Matching tags helped three models, including one from another family.

We tested three models on a shared set of 48-question batches, using different records from slide 7.

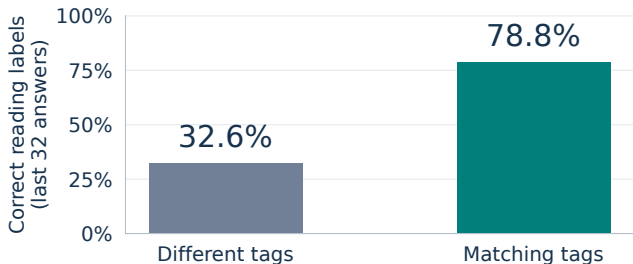


All three improved without retraining. The Mistral model remained less accurate.

Three models, two families, the same 16 batches; 1,536 labels per bar. Software supplied tags and order, not labels. Arbitrary tags helped in every batch for every model. [H3–H5]

Repeating the same tag beside an input and its answer improved accuracy.

We compared the same reading task and record order with matching or different tags.



What changed?

Input tag → answer tag

Different: k7ab → z2pm

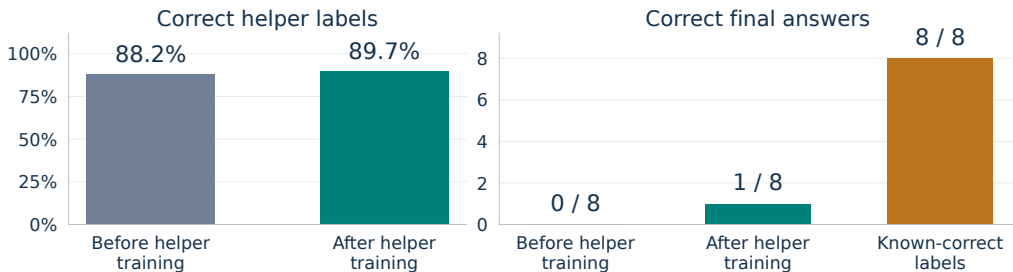
Matching: k7ab → k7ab

Every output followed the required format. Matching helped in all 16 batches.

Illustrative tags; software supplied tags and order, not reading labels. Qwen3-4B, the same 16 batches as slide 8; 3,072 labels per bar. [H6]

Small helper improvements were not enough for reliable final answers.

We trained the helper on question types. Our fixed Python code then combined its labels.



One wrong label can change which users' weights enter the final total.

This is not a matching-tag test. Gold bar: known-correct labels verify the calculation. Eight problems share four inputs; 1,280 helper labels. [C1]

Reward training has not yet improved this trained model.

After example-based training, we rewarded the main model for correct final answers. Its helper stayed fixed.

Before reward
training

55–56

out of 72 questions.

After larger
updates

47–50

out of 72 questions.

After smaller
updates

52–56

out of 72 questions.

Each range shows possible correct final answers, allowing for missing results. Both runs restarted from the same model.

Six weight updates per run. This tests final answers, not slide 5's calculation check. Smaller updates did not clearly change performance. We have not ruled out better reward training. [R1, R2]

The next test is whether these gains help solve a complete problem.

Can the model combine familiar steps in a new way?

The trained routine worked better on new records. We still need to test new combinations of calculations.

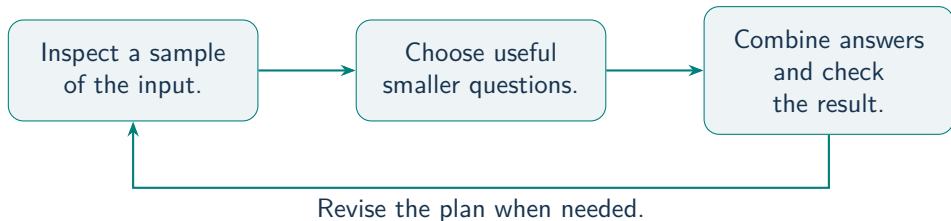
Do better helper answers improve the final answer?

We will change only the matching tags, keep the model and task fixed, and check the complete result.

Better individual answers are a starting point, not the final goal.

These are next tests, not established capabilities. Numbered batch prompts already exist; the contribution must go beyond “numbering helps.”

The longer-term goal is to learn how to investigate and divide a new problem.



We want to solve unfamiliar problems by combining familiar steps. Training and better tools may help the model learn how to choose those steps.

This is a research direction, not an achieved capability. The related `rlm-bootstrap` project supplies ideas for this learning loop. Future comparisons must separate improvements in the model from improvements in its tools.

Which result should we turn into the next focused study?

What would make the finding convincing?

Should we prioritize more model families, a realistic application, or a clearer explanation of why matching helps?

Where would this matter most?

Is there a realistic task in your work where many small judgments must be combined reliably?

Our recommendation is to test whether better input–answer matching improves the final answer on a useful task.

The guide explains each slide. Companion documents give methods, unsuccessful experiments, and possible publication paths.